

Princípios do CQRS

1. **Separação de Responsabilidades:**
 - **Comandos:** Alteram o estado do sistema (escrita).
 - **Consultas:** Lêem o estado do sistema (leitura).
2. **Modelos Independentes:**
 - O modelo de leitura pode ser otimizado para consultas rápidas.
 - O modelo de escrita pode ser mais rigoroso, garantindo consistência.

Benefícios do CQRS

1. **Escalabilidade:** Permite escalar leitura e escrita de forma independente.
2. **Desempenho:** As consultas podem usar um banco otimizado, como cache ou um banco NoSQL.
3. **Flexibilidade:** Facilita a implementação de arquiteturas event-driven (baseadas em eventos).
4. **Simplicidade:** Mantém comandos e consultas focados em suas responsabilidades.

Exemplo Prático de CQRS

Cenário

Um sistema de **Pedidos** precisa:

1. Criar novos pedidos.
2. Consultar detalhes e listas de pedidos rapidamente.

Vamos implementar CQRS separando os modelos de leitura e escrita.

1. Modelo de Domínio (Escrita)

Criamos uma entidade de domínio `Order` que representa o estado do pedido.

```
public class Order
{
    public Guid Id { get; private set; }
    public string CustomerName { get; private set; }
    public List<OrderItem> Items { get; private set; }
    public DateTime CreatedAt { get; private set; }

    public Order(string customerName)
    {
        Id = Guid.NewGuid();
        CustomerName = customerName;
        Items = new List<OrderItem>();
        CreatedAt = DateTime.UtcNow;
    }

    public void AddItem(string productName, int quantity, decimal
unitPrice)
    {
        Items.Add(new OrderItem(productName, quantity, unitPrice));
    }
}

public class OrderItem
{
    public string ProductName { get; private set; }
    public int Quantity { get; private set; }
    public decimal UnitPrice { get; private set; }
    public decimal Total => Quantity * UnitPrice;

    public OrderItem(string productName, int quantity, decimal unitPrice)
    {
        ProductName = productName;
        Quantity = quantity;
        UnitPrice = unitPrice;
    }
}
```

2. Comandos (Escrita)

Criamos comandos para realizar operações no sistema de escrita.

```
public class CreateOrderCommand
{
    public string CustomerName { get; set; }
    public List<CreateOrderItemCommand> Items { get; set; }
}

public class CreateOrderItemCommand
{
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}
```

3. Command Handler

O **Command Handler** executa as operações solicitadas pelos comandos.

```
public class CreateOrderCommandHandler
{
    private readonly IOrderRepository _repository;

    public CreateOrderCommandHandler(IOrderRepository repository)
    {
        _repository = repository;
    }

    public void Handle(CreateOrderCommand command)
    {
        var order = new Order(command.CustomerName);

        foreach (var item in command.Items)
        {
            order.AddItem(item.ProductName, item.Quantity,
item.UnitPrice);
        }

        _repository.Save(order);
    }
}
```

4. Modelo de Leitura

O modelo de leitura é otimizado para consultas rápidas, sem depender diretamente do modelo de escrita.

```
public class OrderReadModel
{
    public Guid Id { get; set; }
    public string CustomerName { get; set; }
    public decimal Total { get; set; }
}
```

5. Queries

As consultas retornam dados diretamente do modelo de leitura.

```
public interface IOrderReadRepository
{
    IEnumerable<OrderReadModel> GetAllOrders();
    OrderReadModel GetOrderById(Guid id);
}
```

Implementação em memória para simulação:

```
public class InMemoryOrderReadRepository : IOrderReadRepository
{
    private readonly List<OrderReadModel> _readModels = new
    List<OrderReadModel>();

    public void Add(Order order)
    {
        _readModels.Add(new OrderReadModel
        {
            Id = order.Id,
            CustomerName = order.CustomerName,
            Total = order.Items.Sum(i => i.Total)
        });
    }

    public IEnumerable<OrderReadModel> GetAllOrders()
    {
        return _readModels;
    }

    public OrderReadModel GetOrderById(Guid id)
    {
        return _readModels.FirstOrDefault(o => o.Id == id);
    }
}
```

5. Uso do CQRS

```
6. class Program
7. {
8.     static void Main(string[] args)
9.     {
10.         // Repositórios
11.         var orderWriteRepository = new
            InMemoryOrderWriteRepository();
12.         var orderReadRepository = new
            InMemoryOrderReadRepository();
13.
14.         // Command Handler
15.         var createOrderHandler = new
            CreateOrderCommandHandler(orderWriteRepository);
16.
17.         // Criar um pedido
18.         var command = new CreateOrderCommand
19.         {
20.             CustomerName = "John Doe",
21.             Items = new List<CreateOrderItemCommand>
22.             {
23.                 new CreateOrderItemCommand { ProductName =
                    "Laptop", Quantity = 1, UnitPrice = 1500 },
24.                 new CreateOrderItemCommand { ProductName = "Mouse",
                    Quantity = 2, UnitPrice = 50 }
25.             }
26.         };
27.
28.         createOrderHandler.Handle(command);
29.
30.         // Atualizar o modelo de leitura (simulação)
31.         var savedOrder = orderWriteRepository.GetLastOrder();
32.         orderReadRepository.Add(savedOrder);
33.
34.         // Consultar pedidos
35.         var orders = orderReadRepository.GetAllOrders();
36.         foreach (var order in orders)
37.         {
38.             Console.WriteLine($"Order ID: {order.Id}, Customer:
                {order.CustomerName}, Total: {order.Total:C}");
39.         }
40.     }
41. }
42.
```

Saída Esperada

```
Order ID: 123e4567-e89b-12d3-a456-426614174000, Customer: John Doe,  
Total: $1,600.00
```

Vantagens do CQRS no Exemplo

1. **Separação de Responsabilidades:** Leitura (`OrderReadRepository`) e escrita (`OrderWriteRepository`) são independentes.
2. **Otimização:** O modelo de leitura é mais simples e direto.
3. **Flexibilidade:** O modelo de leitura pode usar uma abordagem diferente de persistência, como um banco NoSQL.

Conclusão

O **CQRS** é uma poderosa abordagem para sistemas que precisam lidar com grandes volumes de leitura e escrita, mantendo código organizado e eficiente. Ele também facilita a introdução de **event sourcing** para capturar todas as mudanças no estado. □